```
FUNCTION tokenize(infix):
  tokens = []
  FOR each character c in infix:
    IF c is whitespace → skip
    ELSE IF c is digit:
      build full number → add to tokens
    ELSE IF c is letter:
      add variable to tokens
    ELSE IF c is operator or parenthesis:
      add to tokens
  RETURN tokens

FUNCTION precedence(op):
  RETURN precedence value (+,-=1; *,/=2; ^=3)

FUNCTION isLeftAssociative(op):
  RETURN true if op != '^'

FUNCTION infixToPostfix(tokens):
  result = []
  stack = []
  FOR each token:
    IF operand → add to result
    ELSE IF "(" → push to stack
    ELSE IF ")" → pop until "("
    ELSE IF operator:
      WHILE stack not empty AND precedence rules apply:
        pop from stack → add to result
      push operator to stack
```

```
    WHILE stack not empty:
        pop → add to result
    RETURN result

FUNCTION toPostfix(infix):
    tokens = tokenize(infix)
    postfixList = infixToPostfix(tokens)
    RETURN join(postfixList)

FUNCTION toPrefix(infix):
    tokens = tokenize(infix)
    reverse tokens
    swap "(" and ")"
    postfixOfRev = infixToPostfix(reversed tokens)
    reverse postfixOfRev
    RETURN join(postfixOfRev)

MAIN:
    read infix expression from user
    postfix = toPostfix(infix)
    prefix = toPrefix(infix)
    print infix, postfix, prefix
```